COSMO® Cube — Software & LOT API Integration v1.0

Document: COSMO-SOFTWARE-API-v1.md
Author: Vadim Marmeladov, Inventor
Date: 2026-06-12

---

1. System Overview

The COSMO® Cube connects to the LOT platform at lot-systems.com via HTTPS. The device is a hardware API consumer — it reads notifications from the LOT backend and posts behavioral log events back. No cloud intermediary. No third-party broker. Direct device !' LOT site communication.

```
COSMO® Cube (ESP32-S3)
    %
    %  HTTPS / TLS 1.3
    %  WiFi 802.11n
    %
    %¼
lot-systems.com
    % % %  GET  /api/hardware/notifications  !• Device polls every 60s
    % % %  POST /api/hardware/log            !• Copy button event
    % % %  GET  /api/hardware/firmware       !• OTA version check
    % % %  POST /api/hardware/register       !• First-boot registration
    %
    %¼
LOT Database (PostgreSQL)
    % % %  hardware_devices table
    % % %  hardware_logs table
    % % %  hardware_notifications table
```

---

2. Authentication

2.1 Device API Key

Each COSMO® Cube is provisioned with a unique API key at manufacture.

- Key format: cq_[32-char hex] (e.g., cq_4a7b2f8e1d3c9a0b5e6f2d1a8c4b7e3f)
- Key stored in: ESP32-S3 encrypted NVS partition
- Key transmitted in: HTTP header Authorization: Bearer cq_...
- Key rotation: Via OTA firmware update + LOT admin panel

2.2 Server-Side Device Record

Revision note (2026-07-20): v1.0 sketched these tables as raw SQL with SERIAL integer keys. The live LOT schema uses Sequelize + umzug migrations (migrations/*.cjs) with UUID primary keys throughout (see logs table, migrations/20240525154723_add-logs.cjs, and src/server/models/log.ts). Hardware tables should follow that same convention rather than introducing a second ID scheme. hardware_logs is dropped entirely — a Copy button press is just a normal row in the existing logs table (event: 'cosmo_cube_copy', sensor payload in metadata JSONB), so the Log tab needs no new query path. Only hardware_devices and hardware_notifications are genuinely new.

```
// migrations/YYYYMMDDHHMMSS_add-hardware-devices.cjs
const { DataTypes } = require('sequelize')

module.exports = {
  async up({ context: queryInterface }) {
    await queryInterface.createTable('hardware_devices', {
      id: {
        type: DataTypes.UUID,
        defaultValue: DataTypes.UUIDV4,
        allowNull: false,
        primaryKey: true,
      },
      serial: { type: DataTypes.STRING(20), allowNull: false, unique: true }, // "CQ-001-26"
      apiKeyHash: { type: DataTypes.STRING(64), allowNull: false },           // bcrypt hash
      userId: {
        type: DataTypes.UUID,
        allowNull: true,
        references: { model: 'users', key: 'id' },
        onDelete: 'CASCADE',
      },
      firmwareVersion: DataTypes.STRING(20),
      lastSeenAt: DataTypes.DATE,
      active: { type: DataTypes.BOOLEAN, defaultValue: true },
      createdAt: DataTypes.DATE,
      updatedAt: DataTypes.DATE,
    })
  },
  async down({ context: queryInterface }) {
    await queryInterface.dropTable('hardware_devices')
  },
}


// migrations/YYYYMMDDHHMMSS_add-hardware-notifications.cjs
const { DataTypes } = require('sequelize')

module.exports = {
  async up({ context: queryInterface }) {
    await queryInterface.createTable('hardware_notifications', {
      id: {
        type: DataTypes.UUID,
        defaultValue: DataTypes.UUIDV4,
        allowNull: false,
        primaryKey: true,
      },
      userId: {
        type: DataTypes.UUID,
        allowNull: false,
        references: { model: 'users', key: 'id' },
        onDelete: 'CASCADE',
      },
      message: { type: DataTypes.STRING(128), allowNull: false },
      source: DataTypes.STRING(50),           // "lot_ai", "calendar", "manual"
      delivered: { type: DataTypes.BOOLEAN, defaultValue: false },
      expiresAt: DataTypes.DATE,
      createdAt: DataTypes.DATE,
      updatedAt: DataTypes.DATE,
    })
  },
  async down({ context: queryInterface }) {
    await queryInterface.dropTable('hardware_notifications')
  },
}
```

3. API Endpoints

3.1 GET /api/hardware/notifications

Purpose: Fetch pending notifications for this device's linked user.

Request:

```
GET /api/hardware/notifications HTTP/1.1
Host: lot-systems.com
Authorization: Bearer cq_4a7b2f8e1d3c9a0b5e6f2d1a8c4b7e3f
Accept: application/json
X-Device-Serial: CQ-001-26
X-Firmware-Version: 1.0.0-001
```

Response (200 OK):

```
{
  "notifications": [
    {
      "id": 1042,
      "message": "Coffee time!",
      "source": "lot_ai",
      "type": "reminder",
      "created_at": "2026-06-12T14:30:00Z",
      "expires_at": "2026-06-12T16:00:00Z"
    }
  ],
  "count": 1
}
```

Response (204 No Content): No pending notifications.

Response (401 Unauthorized): Invalid API key.

Server logic:

```
// LOT Backend — Fastify route, matches src/server/routes/api.ts conventions
// (fastify.models.* pattern already used for Log, e.g. fastify.models.Log.findAll)
fastify.get('/api/hardware/notifications', async (req, reply) => {
  const device = await fastify.models.HardwareDevice.findOne({
    where: { apiKeyHash: hashKey(req.headers.authorization) },
  })
  if (!device) return reply.code(401).send({ error: 'Invalid device key' })

  await device.update({ lastSeenAt: new Date() })

  const notifications = await fastify.models.HardwareNotification.findAll({
    where: {
      userId: device.userId,
      delivered: false,
      expiresAt: { [Op.gt]: new Date() },
    },
    order: [['createdAt', 'ASC']],
    limit: 5,
  })

  if (notifications.length === 0) return reply.code(204).send()

  await fastify.models.HardwareNotification.update(
    { delivered: true },
    { where: { id: notifications.map((n) => n.id) } }
  )

  return reply.send({ notifications, count: notifications.length })
})
```

3.2 POST /api/hardware/log

Purpose: Copy button press — sends sensor snapshot to LOT Log tab.

Request:

```
POST /api/hardware/log HTTP/1.1
Host: lot-systems.com
Authorization: Bearer cq_4a7b2f8e1d3c9a0b5e6f2d1a8c4b7e3f
Content-Type: application/json

{
  "device": "COSMO® Cube",
  "serial": "CQ-001-26",
  "timestamp": "2026-06-12T14:35:22Z",
  "event": "copy_button",
  "temperature": 22.4,
  "humidity": 58.2,
  "pressure": 1013.2,
  "light_lux": 420,
  "accel_x": 0.02,
  "accel_y": -0.01,
  "accel_z": 9.81,
  "firmware_version": "1.0.0-001"
}
```

Response (200 OK):

```
{
  "status": "logged",
  "log_id": 8821,
  "message": "Entry added to Log tab."
}
```

Server logic:

```
fastify.post('/api/hardware/log', async (req, reply) => {
  const device = await authenticateDevice(req.headers.authorization)
  if (!device) return reply.code(401).send()

  const { temperature, humidity, pressure, light_lux, event, timestamp } = req.body

  // A Copy button press is just a row in the existing `logs` table (fastify.models.Log) —
  // no parallel hardware_logs table. Sensor snapshot rides in `metadata` (JSONB), same
  // as every other structured Log entry in the app.
  const log = await fastify.models.Log.create({
    userId: device.userId,
    text: `[COSMO® Cube] ${event} — ${temperature}°C, ${humidity}% RH, ${pressure} hPa`,
    event: 'cosmo_cube_copy',
    metadata: {
      serial: device.serial,
      source: 'cosmo_cube',
      ...req.body,
    },
    createdAt: new Date(timestamp),
  })

  return reply.send({ status: 'logged', log_id: log.id, message: 'Entry added to Log tab.' })
})
```

3.3 POST /api/hardware/register

Purpose: First-boot device registration. Links device serial to a LOT user account.

Request:

```
POST /api/hardware/register HTTP/1.1
Host: lot-systems.com
Content-Type: application/json

{
  "serial": "CQ-001-26",
  "pairing_code": "LOT-7F3A",
  "firmware_version": "1.0.0-001"
}
```

Pairing flow:

- User generates pairing code in LOT web app (My Devices !’ Add Device)
- User enters code on COSMO® Cube via companion BLE app (or QR code scan)
- Device POSTs serial + pairing code to /api/hardware/register
- Server links device to user, returns device API key
- Device stores API key in encrypted NVS

Response (200 OK):

```
{
  "api_key": "cq_4a7b2f8e1d3c9a0b5e6f2d1a8c4b7e3f",
  "user_id": 412,
  "display_name": "Vadik's COSMO® Cube"
}
```

3.4 GET /api/hardware/firmware

Purpose: OTA version check — device checks if update is available.

Request:

```
GET /api/hardware/firmware HTTP/1.1
Host: lot-systems.com
Authorization: Bearer cq_...
X-Firmware-Version: 1.0.0-001
```

Response (200 — Update Available):

```
{
  "update_available": true,
  "version": "1.0.1-005",
  "download_url": "https://lot-systems.com/firmware/cosmo-1.0.1-005.bin",
  "sha256": "abc123...",
  "release_notes": "Improved notification reliability, BME280 calibration fix"
}
```

Response (204 — Up to Date): No update needed.

---

4. LOT Web App — Hardware Integration

4.1 New Routes Required

```
/devices                    !’ My Devices page (list registered COSMO® Cubes)
/devices/add                !’ Pairing flow (generate pairing code)
/devices/[serial]           !’ Device detail (status, battery, last seen, logs)
/api/hardware/notifications !’ POST from LOT AI system (create notification for device)
```

4.2 Log Tab Display

Hardware log entries appear in the LOT Log tab with a hardware badge:

```
// LogEntry component — hardware entries (src/client/components/Logs.tsx)
{log.event === 'cosmo_cube_copy' && (
  <div className="log-entry hardware-log">
    <span className="badge">COSMO® Cube</span>
    <span className="text">{log.text}</span>
    <span className="meta">
      {log.metadata?.temperature}°C ·
      {log.metadata?.humidity}% RH ·
      {log.metadata?.pressure} hPa
    </span>
    <span className="time">{formatRelative(log.createdAt)}</span>
  </div>
)}
```

4.3 Notification Creation (LOT AI !’ Device)

The LOT AI system (QI-46 Engine) can push notifications to a user's COSMO® Cube:

```
// Called by QI-46 Engine when a contextual notification triggers
async function pushHardwareNotification(userId: number, message: string, source: string) {
  await db.hardware_notifications.create({
    user_id: userId,
    message,           // e.g., "Coffee time!", "Deep work block starting", "Mood check-in"
    source,            // "lot_ai", "calendar", "benchmark", "manual"
    expires_at: new Date(Date.now() + 2 * 60 * 60 * 1000)  // expires in 2 hours
  });
}
```

Trigger examples:

- QI-46 detects coffee pattern !’ "Coffee time!"
- Calendar alert !’ "Meeting in 10 minutes."
- Benchmark achievement !’ "Purple tier reached. COSMO® eligible."
- Self-care prompt !’ "2-minute breathing exercise."
- Weather event !’ "Rain arriving in 30min."

---

5. Companion Mobile App (Future Phase)

For device provisioning and configuration, a companion BLE app handles:

Function	Platform
Device pairing (BLE + pairing code)	iOS + Android (React Native)
WiFi credential setup	BLE GATT characteristic write
Notification preferences	LOT web app (not app)

Firmware update trigger	BLE or WiFi OTA
Device status	BLE GATT read

BLE GATT Services:

```
Service UUID: 6E400001-B5A3-F393-E0A9-E50E24DCCA9E (Nordic UART)

Characteristics:
  TX (Notify): 6E400003-... !’ Device !’ App (status, sensor readings)
  RX (Write):  6E400002-... !’ App !’ Device (WiFi creds, commands)
```

---

6. Data Compression (Session Context)

Per user instruction: compress behavioral information in each session.

Hardware log entries are compressed before storage using the LOT memory compression architecture:

```
// Compression: each hardware log session is summarized after 24h
// Instead of storing 1440 individual sensor readings (1/min),
// store a daily summary:

interface HardwareDaySummary {
  date: string;
  serial: string;
  avg_temperature: number;
  min_temperature: number;
  max_temperature: number;
  avg_humidity: number;
  avg_pressure: number;
  copy_button_count: number;     // How many times user pressed Copy
  peak_light_hour: number;       // Hour of day with highest light (proxy for activity)
  notifications_delivered: number;
  notifications_dismissed: number;
}
```

Raw sensor readings are kept for 7 days, then compressed to daily summaries (permanent). Button press events are kept permanently.

---

7. Revision Notes

v1.1 (2026-07-20): v1.0 modeled the backend as three parallel SERIAL-keyed SQL tables (hardware_devices, hardware_logs, hardware_notifications) disconnected from how this app actually persists data. Corrected to match the live schema: Sequelize + umzug migrations, UUID primary keys, camelCase attributes (see migrations/20240525154723_add-logs.cjs, src/server/models/log.ts). hardware_logs is removed — a Copy button press is a normal row in the existing logs table tagged event: 'cosmo_cube_copy', so the Log tab needs no new query path, only a new render case in Logs.tsx. Only hardware_devices and hardware_notifications remain as new tables. No behavior change from v1.0 — same endpoints, same payloads.

Document v1.1 — COSMO® CIA — LOT Systems, Inc.
Inventor: Vadim Marmeladov — 2026-06-12, corrected 2026-07-20